

Μαντζιούνης Σταμάτιος 4107

Παπανικολάου Νικόλαος 4145

Πετρίδης Ευστάθιος 4157

ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΑΣΚΗΣΗ Ι

Βασικά Αρχεία

Application.cpp

Είναι το αρχείο με το κύριο πρόγραμμα που δημιουργεί το παράθυρο και τρέχει την επανάληψη που ζωγραφίζει τα σχήματα. Εκτός από αυτό περιέχει και την συνάρτηση (`loadTexture()`) που φορτώνει και κάνει map το texture πάνω στην Σφαίρα όπως επίσης και μία συνάρτηση που χειρίζεται τα inputs (`processCameraMovement()`). Οι λειτουργίες των κουμπιών είναι όπως φαίνονται παρακάτω.

Πληκτρολόγιο	Controller (PS/ XBOX)	Λειτουργία
W	R2/RT	Κινεί την κάμερα προς τα Μπροστά
S	L2/LT	Κινεί την κάμερα προς τα Πίσω
A	L Stick	Κινεί την κάμερα προς τα Αριστερά
D		Κινεί την κάμερα προς τα Δεξιά
E		Κινεί την κάμερα προς τα Πάνω
X		Κινεί την κάμερα προς τα Κάτω
I		Γυρίζει την κάμερα να κοιτάει προς τα Πάνω
J	R Stick	Γυρίζει την κάμερα να κοιτάει προς τα Αριστερά
K		Γυρίζει την κάμερα να κοιτάει προς τα Κάτω
L		Γυρίζει την κάμερα να κοιτάει προς τα Δεξιά
Up		Κινεί την Σφαίρα με κατεύθυνση +Y
Right	D-Pad	Κινεί την Σφαίρα με κατεύθυνση -X
Down		Κινεί την Σφαίρα με κατεύθυνση -Y
Left		Κινεί την Σφαίρα με κατεύθυνση +X
NumPad +	L1/LB	Κινεί την Σφαίρα με κατεύθυνση +Z
NumPad -	R1/RB	Κινεί την Σφαίρα με κατεύθυνση -Z
T	 /X	Ενεργοποιεί και απενεργοποιεί το texture

Πληκτρολόγιο	Controller (PS/ XBOX)	Λειτουργία
Space	O/B	Παράγει ένα νέο τυχαίο σωματίδιο
<	Select	Επιβραδύνει τα σωματίδια (με Min = 0)
>	Start	Επιταχύνει τα σωματίδια (MaxSpeedUP = X100)
M	Δ/Υ	Ενεργοποιεί και απενεργοποιεί τους ήχους (windows only)
Esc	X/A	Τερματίζει την εφαρμογή

DynamicShapeArray.h

Είναι το Header του DynamicShapeArray.cpp περιέχει τον ορισμό ενός struct [Shape](#) (περιέχει τις πληροφορίες του κάθε σχήματος), της Κλάσης DynamicShapeArray και των συναρτήσεων της, και κάποια απαραίτητα includes που είναι χρειάζονται σε πολλά αρχεία.

DynamicShapeArray.cpp

Υλοποιεί την κλάση DynamicShapeArray που είναι μία δυναμική λίστα από Shapes και χειρίζεται την δημιουργία, την κίνηση και τις συγκρούσεις τους.

Shader.h

Είναι το Header του Shader.cpp περιέχει τον ορισμό ενός struct [ShaderProgramSource](#) της Κλάσης Shader και των συναρτήσεων της.

Shader.cpp

Υλοποιεί την κλάση Shader που είναι υπεύθυνη για την επικοινωνία του προγράμματος με τους Shaders. Διαβάζει τα προγράμματα από ένα αρχείο (.shader) τα μεταγλωττίζει και μετά χειρίζεται και την αποστολή των uniforms. Είναι βασισμένο στον Shader από το tutorial [\[The Cherno OpenGL Tutorial\]](#).

Shader.shader

Είναι το αρχείο που περιέχει τον πηγαίο κώδικα των Shaders. Δημιουργεί φωτισμό και τοποθετεί τα αντικείμενα στην σκηνή.

Βασικές Λειτουργίες

Δημιουργία Αντικειμένων

Δημιουργούμε αντικείμενα με την `CreateShape()` που δέχεται την θέση που θα τοποθετήσουμε το αντικείμενο (η αρχή του αντικειμένου(0,0,0) όχι το κέντρο του), το μέγεθος του (`d`) και τον τύπο του (`T_CUBE`, `T_SPHERE`, `T_CYLINDER`). Καλεί την κατάλληλη ιδιωτική συνάρτηση, δημιουργεί το αντικείμενο και καλεί την `AddArray()` που το προσθέτει στην `shapeArray` (την χρησιμοποιούμε μόνο για τα 2 αρχικά αντικείμενα). Για τα τυχαία σωματίδια χρησιμοποιούμε την `CreateRandomShape()` που διαλέγει τυχαία τα χαρακτηριστικά των αντικειμένων (εδώ χρησιμοποιήσαμε κάποιες συναρτήσεις της `std`) και χρησιμοποιεί την `CreateShape()` όπως και αντικείμενα που μπαίνουν χειροκίνητα για να τα προσθέσει στην `shapeArray`. Για τους Δακτύλιους η `CreateRandomShape()` καλεί κατευθείαν την `CreateRing()` επειδή χρειάζεται περισσότερα στοιχεία από τα άλλα αντικείμενα που δημιουργούνται από την `CreateShape()`.

Buffers

Δημιουργούμε έναν `VertexArray`, έναν `Vertex Buffer` και έναν `Index Buffer` για κάθε αντικείμενο όταν το δημιουργούμε (το καλεί η `AddArray()` όταν αφού το προσθέσει στην λίστα τους κάνει `generate` (`glGenVertexArrays()` και `glGenBuffers()`), `bind` (`glBindVertexArray()` και `glBindBuffer()`) και μετά περνάει τα κατάλληλα δεδομένα (`Vertex=Shape.data`, `Index=κατάλληλα indices`) και `Attributes` στον `IB` και τον `VB` (`glBufferData()`, `glBufferSubData()`, `glEnableVertexAttribArray()` και `glVertexAttribPointer()`). Τέλος βάζει τα `ids` στα κατάλληλα πεδία του `struct` του Αντικειμένου (`Shape.vao_id`, `Shape.vb_id`, `Shape.ib_id`).

Normals

Επειδή δεν έχουμε περιστροφές τα `normals` για κάθε τύπο σχήματος υπολογίζονται μία φορά (στην πρώτη κλήση των `CreateCylinder()`, `CreateSphere()`, `CreateRing()`, του κύβου είναι `hard coded` στον ορισμό του `cube_normals[]`). Χρησιμοποιούνται για να πετύχουμε σωστό φωτισμό των σχημάτων.

Κίνηση

Σε κάθε σχήμα (`Shape`) κρατάμε ένα `Matrix` (`Shape.Model`) που έχει τους μετασχηματισμούς που έχουν γίνει στην σχήμα αυτό. Στην συνάρτηση `Move()` μεταβάλουμε αυτόν τον πίνακα για κάθε σχήμα σύμφωνα με την ταχύτητα τους (`Shape.speed`) (εάν δεν έχουμε σύγκρουση). Για την Μεγάλη Σφαίρα (`SPH`) χρησιμοποιούμε την συνάρτηση `MoveSphere()` για να την μετακινεί σύμφωνα με την είσοδο του χρήστη.

Συγκρούσεις

Η `Move()` κάθε φορά που καλείται ελέγχει και για σύγκρουση μεταξύ των σχημάτων και τις χειρίζεται η `CheckCollision()/Collide()`. Όταν υπάρξει κρούση υπολογίζουμε την νέα ταχύτητα με βάση την αρχική κατεύθυνση και μέτρο της ταχύτητας των αντικειμένων στο σημείο της κρούσης και παίζει ένα ηχητικό εφέ. Να σημειωθεί ότι οι κρούσεις με δακτυλίους δεν είναι υλοποιημένες σωστά και συμπεριφέρονται περισσότερο σαν κρούσεις με κυλίνδρους.

Φωτισμός

Για τον φωτισμό χρησιμοποιούμε συνδυασμό Ambient, Diffuse και Specular Lighting. Για να το πετύχουμε αυτό περνάμε την θέση του φωτός (`u_Light`) και τα normals(`Normal`) του κάθε σχήματος ως uniforms στον Shader και εκεί υπολογίζουμε το τελικό χρώμα του κάθε σχήματος (`color`).

Textures

Στα textures χρησιμοποιήσαμε την library `stb_image.h` για να φορτώσουμε την εικόνα. Έχουμε την συνάρτηση `loadTexture()` για να φορτώσουμε την εικόνα και να αρχικοποιήσουμε το texture. Χρησιμοποιήσαμε cube maps ώστε να μην χρειάζεται να μετατρέψουμε τις συντεταγμένες σε UV. Στον fragment shader προσθέσαμε κώδικα που συνυπολογίζει το texture και "cubemap sampler" στην περίπτωση που υπάρχει και στον vertex shader απλώς περνάμε τα coordinates (`TexCoords`) του texture (τα οποία λόγω cubemap συμπίπτουν με τα positions του vertex). Στην `main()` του application για να ενεργοποιήσουμε το texture μόνο στην Σφαίρα, ενεργοποιούμε την μεταβλητή `isTexture` και απενεργοποιούμε το bind του texture(`glBindTexture()`).

Extras

Ήδη αναφέρθηκαν οι περισσότερες αλλά τις επαναλαμβάνουμε συγκεντρωμένες εδώ.

- Ήχοι και ηχητικά εφέ
- Επιτάχυνση/Επιβράδυνση σωματιδίων
- Σύγκρουση σωματιδίων
- Δακτύλιοι

Ήχος

Έχουμε προσθέσει και μουσική που παίζει όσο το παράθυρο είναι ανοικτό (2 τραγούδια σε loop με την χρήση του `mciaapi.h`) και ηχητικά εφέ όταν υπάρχει σύγκρουση αντικειμένων (με την `PlaySound()`). (Λειτουργεί μόνο σε windows).

Δακτύλιος

Προσθέσαμε και δακτυλίους στα παραγόμενα σωματίδια επειδή χρειάζεται δύο μεταβλητές για τις διαστάσεις τους ενώ τα υπόλοιπα χρειάζονται μία δεν μπορεί να δημιουργηθεί από την `CreateShape()` χρησιμοποιούμε απευθείας την `CreateRing()`. Ενώ προσθέσαμε και ένα επιπλέον πεδίο στο struct που δεν χρησιμοποιείται από τα υπόλοιπα αντικείμενα (`Shape.d2`). Επειδή υπήρξε δυσκολία στις συνθήκες σύγκρουσης του δώσαμε μία συμπεριφορά παρόμοια με τον κύλινδρο.

Dependencies

Περιέχονται στον φάκελο Dependencies ο κώδικας στον φάκελο Code και τα .lib αρχεία στον φάκελο Libraries.

Από την opengl τοποθετήσαμε στον φάκελο Dependencies\Code τους φακέλους

-GLFW\include\GLFW\

-glm\glm\

-glew\include\GL\

Μέσα στον φάκελο Dependencies\Libraries τοποθετήσαμε τα

-GLFW\lib-vc2019\glfw3.lib

-glew\lib\Release\x64\glew32s.lib

Όστε να τα έχουμε μαζί με τον κώδικα της άσκησης.

Χρησιμοποιήσαμε έκδοση των 64bits.

Όλα αυτά μαζί και με ότι αλλαγές κάναμε στα properties του project υπάρχουν στο αρχείο `dependencies.txt` μέσα στον φάκελο `Dependencies`

Τα αρχεία μουσικής και το texture βρίσκονται μαζί με τον κώδικα της άσκησης.

Προβλήματα-Ακραίες περιπτώσεις

- Εάν δημιουργηθεί ένα αντικείμενο ενώ υπάρχει άλλο αντικείμενο στην θέση δημιουργίας υπάρχει περίπτωση να δημιουργήσει πρόβλημα αν το ένα αντικείμενο χωράει μέσα στο άλλο. Το μικρότερο θα κολλήσει και θα αναπηδά στο εσωτερικό του μεγαλύτερου.
- Η κρούση με δακτυλίους μπορεί να μην γίνεται πάντα σωστά αν και λόγω του μικρού μεγέθους των σωματιδίων δεν προκαλεί πολλά προβλήματα.
- Εάν η ταχύτητα των σωματιδίων είναι πολύ μεγάλη υπάρχει περίπτωση να περάσει μέσα από κάποιο αντικείμενο χωρίς να συγκρουστεί μαζί του. Για αυτό τον λόγο κρατάμε τις ταχύτητες των σωματιδίων σχετικά χαμηλές ακόμα και στην μέγιστη επιτάχυνση.

Βιβλιογραφία

https://www.youtube.com/playlist?list=PLlrATfBNZ98foTJPJ_Ev03o2oq3-GGOS2

[Cherno OpenGL Tutorial YouTube]

https://cgvr.informatik.uni-bremen.de/teaching/cg2_07/literatur/Cube_map_tutorial/cube_map.html?fbclid=IwAR2zlVHh7RR5VpFNGjY1twX_1JY9R8OBwuVAVJOTcom5yX8G5gvx4Ve_6IQ

[Cube Texture Mapping]

http://www.songho.ca/opengl/gl_sphere.html?fbclid=IwAR1oMesRm5wgQLXzM7r90xVPfM_NpZtleTDYbw2wmwhWc34E5IVCkL7I73i4

[Sphere Tutorial Code]

https://www.youtube.com/watch?v=Fw74_fKFowU&fbclid=IwAR0jd0UrVE46VvSQQxFtXEU0jEKSspF0nAHCY3BG_F7VZQHVSkbfsNjgeBg&ab_channel=SurajSharma

[Ambient and Diffuse lighting Tutorial YouTube]

https://learnopengl.com/Advanced-OpenGL/Instancing?fbclid=IwAR3DG2A68Z9wOaLnpOkv1trPCEp6_7mjlkukm4Sw_xOYnXb-9qPPKO_SxgE

[Learn OpenGL: Instancing]

https://www.khronos.org/opengl/wiki/Calculating_a_Surface_Normal?fbclid=IwAR0WQWIG0VTT4Xb0FXn0UqSJCS10slgrdwrR_8xyCieVIGomxdH6JkXAAlI

[Calculating Surface Normals]

<https://learnopengl.com/Lighting/Basic-Lighting?fbclid=IwAR1WLWWgYIOu7mGeYJZ81zuePCPoMWhRkNnb40iGshWun6qHOY2GM6QEgLE>

[Learn OpenGL: Lighting]

[https://docs.microsoft.com/en-us/previous-versions/dd743680\(v=vs.85\)](https://docs.microsoft.com/en-us/previous-versions/dd743680(v=vs.85))

[Playsound documentation]

<https://social.msdn.microsoft.com/Forums/en-US/92803129-24c3-42d0-b32a-17d05606a151/win32-mci-api-for-playing-media?forum=vblanguage>

[MCI API]

<https://stackoverflow.com/questions/22253074/how-to-play-or-open-mp3-or-wav-sound-file-in-c-program>

[Stack Overflow: Play Sounds in programs]

<https://learnopengl.com/Getting-started/Textures>

[Learn OpenGL: Textures]

<https://learnopengl.com/Advanced-OpenGL/Cubemaps>

[Learn OpenGL: Cubemap Texture]

https://www.khronos.org/opengl/wiki/Cubemap_Texture

[Cubemap Texture]